

Práctica 4

Producto de un número

Abigail Sánchez

1. Introducción

Para esta práctica se analizan teórica y experimentalmente cuatro algoritmos para una multiplicación, o, el producto de dos números representados en binario con arreglos de bits. Los tres primeros hacen la multiplicación simulando desplazamientos y sumas, mientras que el cuarto lo calcula mediante sumas acumulativas. EL objetivo es determinar y comparar el tiempo de ejecución de cada uno y evaluar su eficiencia.

2. Marco teórico

Para obtener el producto de dos números binarios lo podemos abordar con distintos métodos:

- 1: simular la multiplicación operando sobre arreglos de tamaño $2n$, desplazando y sumando el multiplicando según los bits del multiplicador.
- 2: optimizar el anterior evitando operaciones cuando el multiplicador es 0.
- 3: no desplazar el arreglo auxiliar, modifica directamente al multiplicando en cada iteración.
- 4: obtener el producto sumando b veces a .

En todos los casos, el análisis se hace considerando arreglos.

3. Análisis de los códigos

Cómo todos los algoritmos comienzan con la misma base del ciclo for, pondré su análisis por separado y solo lo incluiré en las sumas totales de cada algoritmo.

Primero el **ciclo for** con 1 init, una condicion, un incremento y 3 saltos:

$$T_1(n) = 1 + (n + 1) + 2n + 3 = 3n + 5$$

3.1. Código 1 – Algoritmo base

El algoritmo base realiza la multiplicación binaria con arreglos de bits. Su estructura principal es:

```
(1)    for (i = 0; i < n; i++) {
(2)        if (m[i] == 1){
            A = M;
        else
            A = 0;
        }
(3)        A = A << i;
(4)        R = R + A;
    }
```

Analizamos cada bloque por separado:

La condición y el cuerpo del if con 1 acceso a $m[i]$, 1 comparación y un salto; y consideramos 2 escenarios:

- Si el bit es 1: se copian n bits de M a A : n asignaciones
- Si el bit es 0: se ponen en cero los $2n$ bits de A : $2n$ asignaciones

Sea k el número de bits en 1 del multiplicador m . Entonces:

$$T_2(n, k) = 3n + kn + 2n(n - k)$$

Ahora si, el desplazamiento afecta a $2n$ bits y se realiza n veces, con desplazamientos de 0 a $n - 1$:

$$T_3(n) = \sum_{i=0}^{n-1} (2n - i) = 2n^2 - \frac{n(n-1)}{2}$$

Y, la suma de dos arreglos de $2n$ bits con acarreo toman $2n$ operaciones por iteración, y como se ejecuta n veces:

$$T_3(n) = 2n^2$$

Sumando todo quedaría:

$$\begin{aligned}
T(n, k) &= T_1(n) + T_2(n, k) + T_3(n) + T_4(n) \\
&= (3n + 5) + (3n + kn + 2n^2 - 2kn) + (2n^2 - \frac{n(n-1)}{2}) + 2n^2 \\
&= 6n + 5 - kn + 6n^2 - \frac{n(n-1)}{2}
\end{aligned}$$

Y por lo tanto, en el peor caso cuando $k = n$, el orden de la complejidad es:

$$T(n) \in \Theta(n^2)$$

3.2. Código 2 - "Mejora" del base

Modificamos el código 1 para que, si el bit del multiplicador es 0, entonces no se realice.

```

(1)    for (i = 0; i < n; i++) {
(2)        if (m[i] == 1) {
(3)            A = M;
(4)            A = A << i;
(5)            R = R + A;
        }
    }

```

Sean:

- M, m arreglos de n bits; A, R de $2n$ bits.
- k el número de bits en 1 del multiplicador m .
- $S \subseteq \{0, 1, \dots, n-1\}$ el conjunto de índices donde $m[i] = 1$.

La condición y el cuerpo del if, en cada iteración tomamos la condición, salto, acceso y comparación

$$T_1(n) = 3n$$

Copiamos cuando $m[i] = 1$: $A = M$, esto se ejecuta sólo k veces y copia n bits:

$$T_3(n, k) = kn$$

El desplazamiento cuando $m[i] = 1$, es decir, $A = A \ll i$, si A tiene $2n$ bits, en la iteración i cuesta proporcional a $2n - i$, por lo que sólo se ejecuta para $i \in S$:

$$T_4(n, k, S) = \sum_{i \in S} (2n - i) = 2nk - \sum_{i \in S} i$$

Y la suma binaria cuando $m[i] = 1$: $R = R + A$ sobre $2n$ bits, k veces:

$$T_5(n, k) = 2nk$$

Sumando todo quedaría:

$$\begin{aligned} T(n, k, S) &= T_1(n) + T_2(n) + T_3(n, k) + T_4(n, k, S) + T_5(n, k) \\ &= (3n + 5) + 3n + kn + (2nk - \sum_{i \in S} i) + 2nk \\ &= 6n + 5 + kn + 4nk - \sum_{i \in S} i \end{aligned}$$

Como $0 \leq i \leq n - 1$, entonces $n + 1 \leq (2n - i) \leq 2n$. A partir de aquí:

$$6n + 5 + (3n + 1)k \leq T_2(n, k, S) \leq 6n + 5 + kn + 4nk$$

Siendo

- **Peor caso** ($k = n$): $T_2(n) \in \Theta(n^2)$
- **Mejor caso** ($k = 0$): $T_2(n) = 6n + 5 \in \Theta(n)$

Entonces, el algoritmo 2 mantiene el orden cuadrático en el peor caso pero puede mejorar sustancialmente si el multiplicador tiene pocos bits en 1, con una dependencia lineal en k .

3.3. Código 3 - Desplazar M

Para este eliminamos el arreglo auxiliar y desplazamos la multiplicación en cada iteración:

```
(1)    for (i = 0; i < n; i++) {
(2)        if (m[i] == 1){
(3)            R = R + M;
(4)        }
        M = M << 1;
    }
```

Vamos a tener 4 suposiciones:

- m tiene n bits.
- r tiene $2n$ bits.
- Para que podamos sumar sin copiar, vamos a considerar a M llegando a $2n$ bits, es decir que vamos a llenar con 0, así $M \ll 1$ afecta a $2n$ bits, pero, si se mantiene M en n bits, baja a n por iteración. Igual en ambos casos el orden asintótico es el mismo.

- Sea k el número de bits en 1 de m .

Para el if en cada iteración nuevamente se evalúa la condición, que es salto, acceso y comparación.

$$T_2(n) = 3n$$

La suma sobre $2n$ bits, que se ejecuta k veces:

$$T_3(n, k) = 2nk$$

El desplazamiento cuando afecta a $2n$ bits y pasa en cada iteración:

$$T_4(n) = (2n)n = 2n^2$$

Recordemos que si se mantiene M en n , esto sería solo n^2 , la conclusión no cambia.

Y al final si sumamos todo:

$$\begin{aligned} T(n, k) &= T_1(n) + T_2(n) + T_3(n, k) + T_4(n) \\ &= (3n + 5) + 3n + 2nk + 2n^2 \\ &= 6n + 5 + 2nk + 2n^2 \end{aligned}$$

Siendo

- **Peor caso** ($k = n$): $T_3(n) = 6n + 5 + 2n^2 + 2n^2 = 4n^2 + 6n + 5 \in \Theta(n^2)$.
- **Mejor caso** ($k = 0$): $T_3(n) = 2n^2 + 6n + 5 \in \Theta(n^2)$

En general podemos decir que:

$$T_3(n, k) \in \Theta(n^2)$$

3.4. Código 4 - sumas repetidas

Este algoritmo consiste en calcular el producto de dos números a y b mediante una suma acumulativa:

```
for (int i = 0; i < b; i++) {
    resultado += a;
}
```

En cada iteración se realiza:

- Una comparación: $i < b \rightarrow 1$ operación
- Un incremento: $i++ \rightarrow 1$ operación
- Una suma: $resultado += a \rightarrow 1$ operación

Entonces, cada iteración del ciclo realiza 3 operaciones. Si denotamos b como el número de veces que se repite la suma, tenemos:

$$T(n) = 3b + c$$

Donde c es una constante que representa la inicialización.

Para su peor caso, tenemos que b es aleatorio con n bits, en el diseño experimental el valor de b se genera aleatoriamente como un número binario de n bits, es por eso:

$$b \in [0, 2^n - 1]$$

Entonces el peor caso ocurre cuando $b = 2^n - 1$, es decir, cuando todos los bits de b son 1. En ese caso:

$$T(n) = 3(2^n - 1) + c \in \mathcal{O}(2^n)$$

Es decir, la complejidad temporal del código 4 en el peor caso es **exponencial** con respecto al tamaño n de entrada, cuando se representa a b con n bits y se permite que crezca libremente.

La otra opción es el caso controlado donde $b = n$, si se fuerza que el número de repeticiones sea el anterior, el análisis cambia, en este caso el tiempo de ejecución se convierte:

$$T(n) = 3n + c \in \mathcal{O}(n)$$

4. Resultados

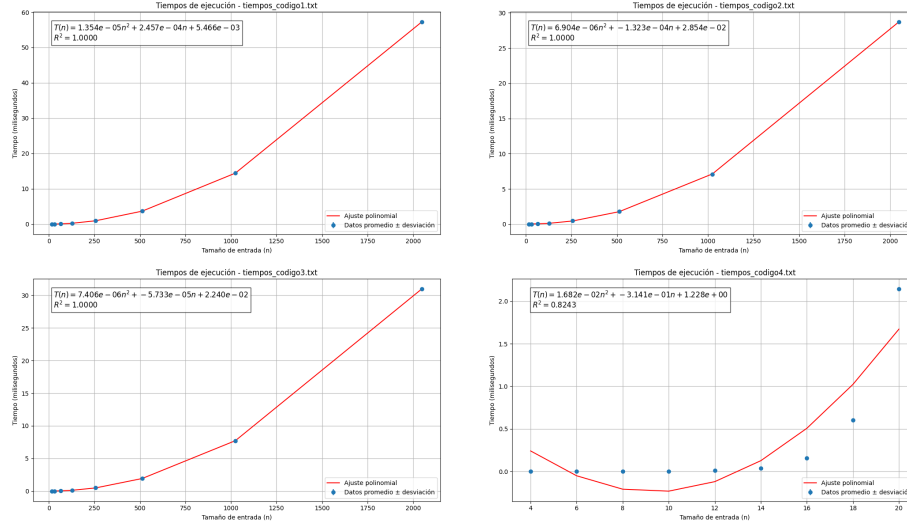


Figura 1: Resultados de los 4 algoritmos implementados

5. Conclusión

Esta práctica permitió comparar 4 algoritmos para obtener el producto de 2 números representados como arreglos de bits. Desde la evaluación teórica y la experimental del tiempo de ejecución, observamos diferencias.

Los primeros 3(que fueron las multiplicaciones binarias con desplazamientos y sumas) presentaron una complejidad asintótica cuadrática, de los cuales, el segundo es ligeramente el más eficiente mejor de estos casos, al evitar operaciones que no son necesarias cuando los bits del multiplicador son 0. El tercero, al eliminar el arreglo y trabajar directamente con el multiplicando desplazado, mantiene el mismo orden pero reduce ciertas asignaciones.

En cambio, el cuarto algoritmo que fueron las sumas acumuladas, depende mucho del valor multiplicado por b. En el mejor de los casos, si b es controlado su complejidad pasa a ser lineal; sin embargo, cuando b puede tomar valores aleatorios de hasta 2^n , el algoritmo se comporta de forma exponencial, entonces lo vuelve ineficiente ante entradas grandes.

Los resultados experimentales confirman las conclusiones teóricas, es importante destacar que tiene su importancia considerar no solo la simplicidad, sino también el comportamiento de los algoritmos con diferentes condiciones al momento de elegir una implementación.